

P2 SM

Objetivo

Crear tu propio sistema de teléfonos como los que usan las empresas, pero usando internet. Eso es exactamente lo que haremos con Asterisk.

¿Qué vamos a construir?

- 2 teléfonos virtuales (sip1 y sip2)
- Buzones de voz para ambos
- Diferentes funciones: llamar, dejar mensajes, conferencias, horarios restringidos

3 archivos principales:

1. pjsip_custom.conf → Lista de teléfonos permitidos
2. voicemail.conf → Configuración de buzones de voz
3. extensions_custom.conf → Qué hace cada número que marcas¹

1-ARCHIVO 1: pjsip_custom_conf (LOS TELÉFONOS)

Aquí pondremos la información de cada usuario que puede usar nuestro sistema.

```
; ===== TRANSPORTE (Cómo viajan las llamadas) =====

[transport_udp]
type=transport           ; Tipo de configuración
protocol=udp             ; Usamos UDP (como cartas normales)
bind=0.0.0.0             ; Interfaz donde se recibirán los datos. Esta in-
dica que se aceptarán por cualquiera

; ===== USUARIO sip1 =====
[sip1]
type=endpoint            ; Esto es un teléfono
aors=sip1                ; Dónde encontrar este usuario
auth=authsip1            ; Clave de acceso
context=practical        ; A qué grupo pertenece
mailboxes=001@sm         ; Su buzón de voz
disallow=all             ; Primero prohibimos todo
allow=alaw,g723          ; Luego permitimos estos formatos de audio
transport=transport_udp  ; Usa el transporte que definimos arriba

[authsip1]
type=auth                ; Configuración de contraseña
auth_type=userpass       ; Usuario y contraseña
password=passwdsip1      ; Contraseña de sip1
username=sip1            ; Nombre de usuario

[sip1]
type=aor                 ; Dirección de contacto
max_contacts=1           ; Máximo 1 teléfono por usuario
remove_existing=yes      ; Si se conecta otro, quita el anterior

; ===== USUARIO sip2 (TODO IGUAL PERO PARA sip2) =====
[sip2]
type=endpoint
aors=sip2
auth=authsip2
context=practical
mailboxes=002@sm
disallow=all
allow=alaw,g723
transport=transport_udp

[authsip2]
type=auth
auth_type=userpass
password=passwdsip2
```

```
username=sip2  
  
[sip2]  
type=aor  
max_contacts=1  
remove_existing=yes
```

Ahora tenemos que configurar en la misma red en dos dispositivos en el Linphone a los dos usuarios sip1 y sip2.

Por ejemplo:

En el ordenador:

Sip1 -> usuario sip1, contraseña passwdsip1, dominio IP de la red y transporte UDP.

Sip2 -> usuario sip2, contraseña passwdsip2, dominio IP de la red y transporte UDP.

Para comprobar que funciona, en la consola de Asterisk:

```
pjsip show endpoints
```

2-ARCHIVO 2: pjsip_custom_conf (LOS BUZONES DE VOZ)

Aquí ponemos los buzones donde se guardan los mensajes cuando no contestas.

```
; ===== CONFIGURACIÓN GENERAL =====
[general]
format=wav49|wav          ; Formato de los mensajes de voz
serveremail=voicemail@udc.test.es ; Email que envía los mensajes
attach=yes                ; Adjuntar audio en emails
maxsilence=10             ; 10 segundos de silencio = fin mensaje
maxlogins=3               ; Máximo 3 intentos de contraseña

; ===== ZONAS HORARIAS =====
[zonemessages]
central=America/Chicago|'vm-received' Q 'digits/at' IMP
european=Europe/Copenhagen|'vm-received' a d b 'digits/at' HM

; ===== BUZONES DE LOS USUARIOS =====
[sm] ; Este es el contexto "sm" que usamos en pjsip_custom.conf
; Formato: número => PIN, Nombre, Email, Email_corto, Opciones
001 => 0000, Ruben, ruben.gonzalez.ouzonis@udc.es,,
002 => 0000, Ruben, ruben.gonzalez.ouzonis@udc.es,,
```

[general]

Incluye parámetros generales de configuración.

se podría poner:

mailcmd: permite poner un comando para enviar los emails diferente de /usr/sbin/

sendmail -t

[zonemessages] es opcional

[sm] es el contexto del buzón de voz. Dentro se definen los buzones de voz:

Explicación del formato de buzón de voz:

- mailbox: 001 → Número del buzón (igual que en mailboxes=001@sm)
- PIN: 0000 → PIN para escuchar mensajes (4 dígitos)
- Name: Ruben → Nombre del dueño
- Email_Address: ruben.gonzalez.ouzonis@udc.es → Email para notificaciones
- Short_email_address: (vacío) → Email corto

Se pueden poner opciones con | al final de cada buzón:

112 => 0000,Pepe,pepe@udc.test.es,,tz=eastern|maxmsg=5

Aquí tiene como límite 5 mensajes el buzón de voz.

Para comprobar que funciona, en la consola de Asterisk:

voicemail reload

voicemail show users

3-ARCHIVO 3: extensions custom.conf (QUE HACE CADA NÚMERO)

Este archivo se encarga de configurar el dialplan. Un dialplan es un **conjunto de reglas** que le indican al sistema telefónico cómo debe manejar e interpretar la secuencia de dígitos que marca un usuario.

-Contextos:

Un dialplan se divide en secciones, llamados **contextos** dentro de Asterisk

```
1 [nombre_de_contexto]
2 ; Unas acciones
3
4 [otro_contexto]
5 ; Otras acciones
6 ...
```

- Extensiones (dentro de contextos)

Dentro de cada contexto se definen una o más extensiones. Una extensión define una acción o serie de acciones que se ejecutan cuando un cliente marca su número asociado.

- Una extensión se define como:

```
1 exten => numero_de_extension, prioridad, aplicacion
```

- Numero extension: lo que tiene que marcar el cliente para que se ejecute la extensión.
- Prioridad: cuando la extensión se compone de varias acciones, cada una debe llevar asociado un número consecutivo que indica el orden en el que se deben ejecutar
- Aplicación: son las operaciones que se ejecutan en cada paso de la extensión.

Aplicaciones más importantes:

Para llamadas:

- Dial(PJSIP/sip1,30) → Llama a sip1 por 30 segundos
- Hangup() → Cuelga la llamada
- Answer() → Contesta

Para sonidos:

- Playback(hello-world) → Reproduce un audio
- SayDigits(123) → Dice "uno dos tres"
- SayNumber(123) → Dice "ciento veintitrés"

Para leer datos:

- Read(digitos,beep,4) → Lee hasta 4 dígitos después de un beep

Para decidir:

- GotoIf(condición?si:no) → Si condición verdadera va a "si", sino a "no"

Ejemplo simple:

exten => 100,1,Answer()	; Prioridad 1: Contesta
same => n,Playback(hello-world)	; Prioridad 2: Dice "hola mundo"
same => n,Hangup()	; Prioridad 3: Cuelga

La prioridad n significa +1 que la prioridad anterior.

3.1-VARIABLES

Permiten guardar valores y utilizarlos después en el dialplan:

- Se accede con la sintaxis `${MiVariable}`.
- Case sensitive: distingue mayúsculas de minúsculas.
- Dependiendo del ámbito de validez, las variables pueden ser globales, de canal, o predefinidas de Asterisk.

1- Variables **globales**: Accesibles desde cualquier canal en cualquier momento (para todos). Se definen en el contexto **[globals]**:

```
1 [globals]
2 UserA_DeskPhone=PJSIP/0000f30A0A01
3 UserA_SoftPhone=PJSIP/SOFTPHONE_A
4
5 [other]
6 exten => 100,1,Dial(${UserA_DeskPhone})
```

2- Variables de **canal**: Solo existen durante la llamada (solo para esta llamada). Se definen con la aplicación **Set()**:

```
1 exten => 203,1,Noop(say some digits)
2 same => n,Set(SomeDigits=123)
3 same => n,SayDigits(${SomeDigits})
```

3- Variables **predefinidas de Asterisk**: asociadas a Asterisk, se generan automáticamente y se accede a ellas mediante su nombre reservado, de forma similar a cualquier variable. Contienen información útil que se puede utilizar para definir las extensiones. Por ejemplo:

- **CONTEXT**: Nos dice el cual es el contexto actual.
- **EXTEN**: Nos dice cual es el número de extensión actual.
- **PRIORITY**: Nos dice cual es la prioridad actual.
- **UNIQUEID**: Nos da el identificador único de canal generado por Asterisk. P.ej: 1267568856.11.
- **CHANNEL**: Nos da el nombre del canal generado por Asterisk. P.ej: PJSIP/sip1-00000010.

(Nota: CHANNEL es también una función, se define en la sección Funciones).

TRUCOS IMPORTANTES:

-Para juntar variables:

`${Parte1}${Parte2}` ; → "HolaMundo"

-Cadenas de caracteres sin comillas:

`Set(Texto=Hola)` ;  Bien

`Set(Texto="Hola")` ;  Sale "Hola" con comillas

-Duración de variables:

`Set(Normal=valor)` ; Solo aquí

`Set(_UnaMas=valor)` ; Dura un salto más

`Set(__Siempre=valor)` ; Dura para siempre

Ejemplos:

Caso1:

```
exten => 100,1,Set(Mensaje=Hola) ; Escribes "Hola"
same => n,Dial(PJSIP/sip1) ;  Aquí sí existe
same => n,SayDigits(${Mensaje}) ;  Aquí YA NO existe - se perdió
```

Caso2:

```
exten => 100,1,Set(_Mensaje=Hola) ; Escribes "Hola" con _
same => n,Dial(PJSIP/sip1) ;  Pasa al siguiente
same => n,SayDigits(${Mensaje}) ;  Aquí SÍ existe
same => n,Dial(PJSIP/sip2) ;  Aquí YA NO existe
```

Caso3:

```
exten => 100,1,Set(__Mensaje=Hola) ; Escribes "Hola" con __
same => n,Dial(PJSIP/sip1) ;  Sigue
same => n,Dial(PJSIP/sip2) ;  Sigue
same => n,Dial(PJSIP/sip3) ;  Sigue siempre
```

-Cortar palabras:

```
Set(palabra=TELEFONO)
```

```
${palabra:1:3} ; → "ELE"
```

```
; Empieza en posición 1 (E), toma 3 letras: E-L-E
```

```
${palabra:0:4} ; → "TELE"
```

```
; Empieza en posición 0 (T), toma 4 letras: T-E-L-E
```

```
${palabra:-1:3} ; → "O"
```

```
; Empieza en la última letra (O), toma 3 letras (solo hay 1): O
```

```
${palabra:2:-2} ; → "LEF"
```

```
; Empieza en posición 2 (L), quita 2 letras del final
```

```
; "TELEFONO" → quita "NO" → "TELEFO" → desde L: "LEF"
```

\${numero}="98765":

– \${numero:1:3} sería "876"

Empieza en índice 1 (8), toma 3 caracteres

– \${numero:-1:3} sería "5"

Empieza en último carácter (5), toma 3 → solo hay 1 → 5

– \${numero:1:-3} sería "8"

Empieza en índice 1 (8), "toma hasta 3 antes del final" → solo 8

– \${numero:-1:-3} sería ""

nada, vacío

Empieza en la última 5 y descarta las tres 3 últimas del final ->

3.2-EXPRESIONES

Permiten comparar variables y hacer operaciones con ellas.

- Sintaxis: `$(expresion)`. Ej. `...Set(NEWCOUNT=$(COUNT) + 1)`
- Los valores numéricos se toman como flotantes.
- `|` or
- `&` and
- `=`, `>`, `>=`, `<`, `<=`, `!=` comparadores numéricos o de strings según el tipo de variable.
- `+`, `-`, `*`, `/`, `%` operadores matemáticos.
- `:` compara dos expresiones regulares, suponiendo que la expresión regular ya lleva un `^` implícito indicando que se tiene que machear desde el principio del string.

Para evaluar cosas matemáticamente tienen que ir entre corchetes `[$[]`.

3.3 PATTERN MATCHING

En lugar de crear una extensión para cada número, creas **patrones** que sirven para varios números a la vez. Es decir, antes vimos que una extensión poníamos un número único que se podía usar para llamar, pues ahora indicaremos un rango con pattern mathching de modo que podamos con una extensión llamar a varios números a la vez.

- Todos los patrones empiezan con _ (guión bajo)
- Normalmente se usan números, pero podrían ser letras.
- Caracteres especiales:
 - X: es cualquier número entre 0 y 9.
 - Z: lo mismo entre 1 y 9.
 - N: lo mismo entre 2 y 9.

Ejemplos:

_64XX ; Vale para: 6400, 6401, 6402... hasta 6499

_1NX ; Vale para: 120, 121, 122... 130, 131... 190, 191...

RANGOS:

Pones entre corchetes [] los números que quieres que valgan:

_6[123]X ; Vale para: 610, 611... 620, 621... 630, 631...

_[2-5]XX ; Vale para: 200-599 (cualquier número de 200 a 599)

[2-5]: del 2 al 5

[1-468]: del 1 al 4 y 6 y 8 solo. Solo uno de ellos.

COMODINES ESPECIALES:

_6. ; Vale para: 6, 60, 61, 600, 6123... (cualquier cosa que empiece con 6)

_6! ; Igual que arriba

COINCIDENCIA DE PATRONES:

Cuando un número puede entrar en varios patrones, se usa este orden de prioridad:

1. El más específico gana
2. Menos opciones gana sobre más opciones
3. Números exactos ganan siempre

Ejemplo: Para el número 6421

- `_642X` gana sobre `_64XX`
- Porque `_642X` es más específico

Ejemplo completo:

`exten => _640X,1,GoTo(C)` ,6401 a 6409. Si marcan un número entre esos va a C.

`exten => _64NX,1,GoTo(E)` ,6420 a 6499. Si marcan un número entre esos va a E.

`exten => _64XX,1,GoTo(B)` ,6400 a 6499. Si marcan un número entre esos va a B.

`exten => _6[34]NX,1,GoTo(G)` ,6320 a 6499. Si marcan un número entre esos va a G.

`exten => _6[45]NX,1,GoTo(F)` ,6420 a 6599. Si marcan un número entre esos va a F.

`exten => _6XX1,1,GoTo(A)` ,6001 a 6991. Si marcan un número entre esos va a A.

`exten => _6.,1,GoTo(D)` , cualquier número que empiece en 6. Va a D.

Que pasa si llamamos al 6421?

1.  `_640X` - No coincide (no es 640X)
2.  `_64NX` - Coincide (64 + N=2 + X=1) → **ESTE GANA**
3.  `_64XX` - Coincide (64 + X=2 + X=1)
4.  `_6[34]NX` - Coincide (64 está en [34]? No, pero 64 está en el rango?)

- 5. ☒ _6[45]NX - Coincide (64 está en [45])
- 6. ☐ _6XX1 - No coincide (no termina en 1)
- 7. ☒ _6. - Coincide (empieza con 6)

Gana _64NX por ser el más específico de los que coinciden.

En la consola de Asterisk, se pueden ver las extensiones que coinciden con una dada: dialplan **show 6421@micontexto** mostraría que extensiones del contexto micontexto se activarían al recibir el código 6421. Las extensiones se muestran ordenadas.

3.4 APLICACIONES

Son las operaciones que se ejecutan en cada paso de la extensión.

3.4.1- Aplicaciones de llamadas

- **Answer()** - Contesta la llamada
- **Hangup()** - Cuelga la llamada
- **Dial()** - Hace una llamada

Dial(PJSIP/sip1,30,m) ; Llama a sip1, 30 segundos, música de espera (solo es obligatorio el primer parámetro).

Tras usar Dial() para hacer una llamada, el resultado de la misma se almacena en la variable de entorno DIALSTATUS. Valores típicos de esa variable:

1. ANSWER : Se respondió la llamada.
2. BUSY: El endpoint está ocupado con otra llamada o el dispositivo devolvió ese estado.
3. NOANSWER: Se llamó hasta que saltó el timeout y no se obtuvo respuesta.
4. CANCEL: el cliente colgó la llamada antes de contestar.
5. INVALIDARGS: Algún error en los argumentos de Dial().
6. CHANUNAVAIL: El endpoint no hizo el registro y Asterisk no sabe como encontrarlo.

- **Wait(5)** - Espera 5 segundos
- **WaitExten(10)** - Espera que marquen una extensión (10 segundos)

3.4.2- Aplicaciones de reproducir sonidos

- **Playback(hola)** - Reproduce audio "hola"
- **SayDigits(123)** - Dice "uno dos tres"
- **SayNumber(123)** - Dice "ciento veintitrés"

- **SayAlpha(abc)** - Dice "a b c"
- **Background(menu)** - Reproduce audio pero permite interrumpir con teclas

3.4.3- Aplicaciones para leer datos de usuarios

- **Read()** - Lee dígitos del teléfono

Read(variable, audio, max_digitos, opciones, intentos, tiempo)

Tras usar Read() para leer dígitos en una llamada, el resultado de la misma se almacena en la variable de entorno READSTATUS. Valores típicos de esa variable:

1. OK: Leyó bien los dígitos
2. ERROR: error en la lectura
3. HANGUP: Usuario colgó
4. INTERRUPTED: lectura interrumpida
5. SKIPPED: lectura omitida
6. TIMEOUT: Se acabó el tiempo

3.4.4- Aplicaciones de control de flujo

- **Goto()** - Salta a otra parte

Goto(100,1) ; Salta a extensión 100, prioridad 1

Goto(otrocontexto,200,1) ; Salta a otro contexto

- **GotoIf()** - Salto condicional

GotoIf[\${edad} >= 18]?mayor:menor) ,Si edad mayor o igual que 18 salta a mayor, si no a menor.

- **GotoIfTime()** - Salto por horario

GotoIfTime(09:00-17:00,mon-fri,*. *?oficina:fuera)

- **While()** - Bucles

While(\$[\${contador} > 0])

...

EndWhile()

3.4.5- Subrutinas

Las subrutinas son funciones que cualquier contexto puede usar. Se definen igual que los contextos. Es un código reutilizable usado por diferentes contextos.

Los contextos usan las subrutinas usando la aplicación **GoSub**:

- **GoSub**: Llama a una subrutina

GoSub(misubrutina,start,1(param1,param2))

- **GoSubIf**: sería un salto condicional a una subrutina

Tras usar GoSub() para ejecutar una subrutina, el resultado de la misma se almacena en la variable de entorno GOSUB_RETVAL. Valores típicos de esa variable:

1. Números: 15, 0, -5
2. Texto: "exito", "error", "ocupado"
3. Estados: ANSWER, BUSY, NOANSWER

3.4.6- Aplicaciones para debug

- **Verbose()** - Mostrar mensajes en consola

Verbose(1,El usuario marcó \${EXTEN})

- **Noop()** - No hace nada, útil para comentarios

3.4.7- Aplicaciones relacionadas con el buzón de voz

VoiceMail() - Dejar mensaje de voz

se puede dejar el mensaje en varios buzones a la vez. Si se especifican varios buzones, el saludo que se muestra será el especificado en el primero, y si alguno de la lista no existe, el dialplan se para. Entre las opciones tendríamos:

- b: reproduce el saludo de ocupado (busy).
- u: reproduce el saludo de no disponible (unavailable)(comportamiento por defecto).
- d([contexto]): permite aceptar dígitos de una nueva extensión cuando se está escuchando

el mensaje de bienvenida. contexto es un parámetro opcional, si no se pone nada se usa el

contexto actual.

- s: omite las instrucciones de cómo dejar un mensaje tras el saludo.
- U: marca el mensaje como URGENT. Después cuando el usuario consulte sus mensajes, se le informará de que este era urgente.
- P: marca el mensaje como PRIORITY.

Tras usar VoiceMail() para dejar mensajes en el buzón, el resultado de la misma se almacena en la variable de entorno **VMSTATUS**. Valores típicos de esa variable:

1. SUCCESS = mensaje dejado
2. USEREXIT = usuario colgó
3. FAILED = error

- **VoiceMailMain()** - Escuchar mensajes

Variables Importantes

\${DIALSTATUS} - Resultado de Dial() (ANSWER, BUSY, NOANSWER)

\${READSTATUS} - Resultado de Read() (OK, TIMEOUT, ERROR)

\${GOSUB_RETVAL} - Valor devuelto por subrutina

\${VMSTATUS} - Resultado de VoiceMail() (SUCCESS, FAILED)

3.5 FUNCIONES

Son herramientas para trabajar con datos (texto, números, fechas, etc.)

Sintaxis: `${NOMBRE_FUNCION(parametro1, parametro2)}`

Funciones útiles:

- **LEN()** - Longitud de texto

`${LEN(hola)}` ; = 4 (4 letras)

`${LEN(12345)}` ; = 5 (5 números)

- **ISNULL()** - Ver si está vacío

`${ISNULL(${variable})}` ; = 1 si está vacía, 0 si tiene valor

- **IF()** - Decisión

`${IF(${edad} >= 18?mayor:menor)}` ; Si edad >=18 → "mayor", sino "menor"

- **RAND()** - Número aleatorio

`${RAND(1,100)}` ; Número entre 1 y 100

- **CALLERID()** - Información de quien llama

`${CALLERID(num)}` ; Número de teléfono del que llama

`${CALLERID(name)}` ; Nombre del que llama

- **CHANNEL()** - Información del canal

Un canal es la conexión de una llamada telefónica. Cada llamada es un canal nuevo.

Ejemplo:

sip1 —————→ Asterisk —————→ sip2

(canal PJSIP/sip1-123) (canal PJSIP/sip2-456)

PJSIP/sip1-0001 ← Canal de sip1 (llamada #0001)

PJSIP/sip2-0002 ← Canal de sip2 (llamada #0002)

DAHDI/1-1234 ← Canal de línea telefónica normal

`${CHANNEL(endpoint)}` ; sip1, sip2 (quién llama)

`${CHANNEL(exten)}` ; Número marcado (100, 200)

`${CHANNEL(context)}` ; Contexto actual

3.6 CANALES LOCALES

Sirven para llamar a varios teléfonos EN SECUENCIA (uno tras otro) con diferentes tiempos. Queremos llamar a 2 teléfonos pero no al mismo tiempo. Primero sip1, si no contesta → luego sip2

Ejemplo:

[principal]

exten => 100,1,Dial(Local/fijo@llamadas & Local/celular@llamadas,30)

[llamadas]

; Teléfono FIJO (llama inmediatamente)

exten => fijo,1,Dial(PJSIP/telefono-fijo)

; CELULAR (espera 10 segundos antes de llamar)

exten => celular,1,Wait(10)

same => n,Dial(PJSIP/celular)

Una vez acabada la teoría vamos a completar el archivo de **extensions_custom.conf**. Esto es lo que tiene que hacer nuestro dialplan:

01 Permite llamar al *endpoint* sip1.

02 Permite llamar al *endpoint* sip2.

Para hacer esto necesitaremos las subrutinas **sub_recordcallinfo** y **sub_sipdial**

00X Con **X** = 1 o 2. Permite llamar a sip1 o sip2, y si no se puede entonces activa el buzón de voz para poder dejar un mensaje. La secuencia de acciones seria:

1. Reproducir el mensaje *you-have-dialed*.
2. Decir la extension marcada.
3. Llamar a sip1 si **X** = 1; llamar a sip2 si **X** = 2.
4. Si no se puede realizar la llamada hacer saltar el buzón de voz. El mensaje de bienvenida debe corresponderse con la causa que provoco que no se pudiese realizar la llamada: *busy* si la llamada finalizo con un estatus de ocupado o *unavailable* en otro caso.

Para hacer esto necesitaremos la subrutina **sub_voicemail**

120 El usuario que llama puede consultar su propio buzón de voz. Detecta automáticamente quien llama y abre su buzón correspondiente.

99X Con **X** = 1 o 2. Permite llamar a sip1 o sip2, pero solo en unas franjas horarias determinadas.

La extension debe **aceptar** las llamadas solo en el siguiente horario:

Lunes a Jueves de 15:30 a 20:00, durante los todos los meses del año excepto en Agosto.

Si se llama fuera del horario permitido se reproducira un mensaje (e.g., *unavailable & please-try-again-later*). Si se llama dentro del horario permitido, se llamara a sip1 si **X** = 1, o a sip2 si **X** = 2.

300 Permite al usuario conectarse a una sala de conferencia, introduciendo antes un PIN de cuatro digitos: 4323. Si no lo introduce correctamente antes de tres intentos, la llamada finalizara. En caso de introducir el pin correcto, el usuario será conectado a la sala de conferencia numero 7007.

Para hacer esto necesitamos la subrutina **sub_conference**.

#0 Se reproduce un menu con el siguiente comportamiento:

1. Se le pide al usuario que pulse 0, 1, 2, o 3 (e.g., Playback(press) & SayNumber(1) & Playback(or) . . .).
2. Se espera a que marque la extension (maximo 10 segundos).
3. Se realiza una accion dependiendo de la extension marcada:

0 Se indica el numero de llamadas que no se han podido enviar por estar no disponible, y al acabar se vuelve al punto 1.

1 Se indica el numero de llamadas en las que estaba ocupado, y al acabar se vuelve al punto 1.

2 Se indica el numero de llamadas que ha contestado, y al acabar se vuelve al punto 1.

3 Se reproduce el mensaje *goodbye* y se cuelga la llamada.

4. Si la extension marcada no es valida reproducir el mensaje *pbx-invalid* y se vuelve al punto 1.

5. Si se agota el tiempo de espera sin marcar ninguna extension se reproduce el mensaje *goodbye* y se cuelga la llamada.

Ejercicio 3: Implementar la subrutina **sub_recordcallinfo**

Implementa la subrutina **sub_recordcallinfo** a partir del código 3 como sigue:

1. Completa el fragmento resaltado (a) con el resto de posibles valores de **DIALSTATUS**.
2. Completa el fragmento resaltado (b) con el código para crear una variable local llamada **familykey**, que contendrá la cadena *familia/clave* para AstDB, con el valor **calls_usuario/clave**, siendo **usuario** el indicado en el parámetro **ARG1** de la subrutina y **clave** la establecida en la variable local **key**.
3. Completa el fragmento resaltado (c) con el código para asignar en AstDB el valor de **count + 1** a la familia/clave contenida en la variable local **familykey**.

ESTA SUBROUTINA GUARDA INFORMACIÓN SOBRE EL ESTABLECIMIENTO DE LAS LLAMADAS EN UNO DE LOS DOS USUARIOS (sip1 o sip2).

Tenemos que guardar en la base de datos si las llamadas fueron contestadas (a-answer), ocupadas (b-busy), no disponibles (u-chanunavail), no contestadas (n-noanswer) u otro (o-other).

1-

```
same => n,ExecIf($[{$ARG2}=BUSY]?Set(LOCAL(key)=b))
```

```
same => n,ExecIf($[{$ARG2}=CHANUNAVAIL]?Set(LOCAL(key)=u))
```

```
same => n,ExecIf($[{$ARG2}=NOANSWER]?Set(LOCAL(key)=n))
```

Convierte estados de llamada en letras para la base de datos.

- Usa las variables **\${ARG2}** (parámetro que recibe la subrutina (estado de llamada)) y **LOCAL(key)** (variable temporal dentro de la subrutina).

- Usa la expresión de = para comparar si el **\${ARG2}** es igual a los estados de la variable del **DIALSTATUS**.

- Usa las aplicaciones **ExecIf()** que ejecuta condicionalmente y **Set()** que asigna un valor a una variable en este caso a **LOCAL(key)**.

2-

```
same => n,Set(LOCAL(familykey)=calls_{$ARG1}/${LOCAL(key)})
```

Construye la dirección exacta donde guardar en la base de datos AstDB.

- Usa las variables `${ARG1}` (parámetro de la subrutina (usuario: sip1 o sip) , `LOCAL(key)` (variable temporal con la letra (b, u, n, a, o)), `LOCAL(familykey)` (nueva variable que se crea).
- Usa concatenación `calls_${ARG1}/${LOCAL(key)}` para juntar texto y variables.
- Usa la aplicación Set para asignar un valor a una variable.

3-

same => n,Set(DB(\${LOCAL(familykey)})=\${LOCAL(count)} + 1)

Suma +1 al contador de llamadas del tipo específico en la base de datos.

- Usa las variables `LOCAL(familykey)` (Ruta de base de datos (ej: calls_sip1/b)), y `LOCAL(count)` (número actual de llamadas).
- Usa las aplicaciones Set() para asignar valores y DB() que es la función de la base de datos AstDB.
- Usa la expresión `${LOCAL(count)} + 1` que lo que hace es sumar 1 al contador actual de llamadas de cada tipo.

```
[sub_recordcallinfo]
; ARG1 = usuario (sip1), ARG2 = estado (ANSWER, BUSY, etc.)
exten => start,1,Verbose(3,Recording call info: ${ARG1} (${ARG2}))

; Convertir estado a letra:
same => n,ExecIf(${LOCAL(ANSWER)}?Set(LOCAL(key)=a)) ; a = contestada
same => n,ExecIf(${LOCAL(BUSY)}?Set(LOCAL(key)=b)) ; b = ocupado
same => n,ExecIf(${LOCAL(CHANUNAVAIL)}?Set(LOCAL(key)=u)) ; u = no disponible
same => n,ExecIf(${LOCAL(NOANSWER)}?Set(LOCAL(key)=n)) ; n = no contestó

; Si no es ninguno:
same => n,ExecIf(${ISNULL(LOCAL(key))}?Set(LOCAL(key)=o))

; Crear dirección en base de datos: calls_sip1/a, calls_sip1/b, etc.
same => n,Set(LOCAL(familykey)=calls_${ARG1}/${LOCAL(key)})

; Obtener valor actual y sumar 1
same => n,Set(LOCAL(count)=${DB(LOCAL(familykey))})
same => n,ExecIf(${ISNULL(LOCAL(count))} = 1)?Set(LOCAL(count)=0)
same => n,Set(DB(LOCAL(familykey))=${LOCAL(count)} + 1)

same => n,Return()
```

Ejercicio 4: Implementar la subrutina `sub_sipdial`

Implementa la subrutina `sub_sipdial` a partir del código 4 como sigue:

1. En el fragmento resaltado (a) usa la aplicación `Dial()` para llamar al *endpoint* indicado en el parámetro `ARG1`, con el *timeout* indicado en el parámetro `ARG2`, y las opciones indicadas en el parámetro `ARG3`.
2. Si la llamada realizada por la aplicación `Dial()` definida en el fragmento (a) no se responde (por *timeout* o por alguna otra razón) se continuará con el *dial-plan* en la extensión actual. En el fragmento resaltado (b), llama a la subrutina `sub_recordcallinfo`, pasándole como parámetros el *endpoint* indicado en el parámetro `ARG1` y el estado de la llamada indicado en `DIALSTATUS`.
3. Si la llamada se contesta, al finalizarla se saltará a la extensión especial *h*. Completa el fragmento (c) para que, cuando esa extensión se ejecute, se salte a la prioridad etiquetada como *record*, y que así se guarde en la base de datos también las veces que se contestó la llamada.

ESTA SUBROUTINA LLAMA A ALGUIEN Y AUTOMÁTICAMENTE GUARDA LO QUE PASÓ.

ARG1=usuario, ARG2=tiempo, ARG3=opciones

1-

same => n,Dial(PJSIP/\${ARG1},\${ARG2},\${ARG3})

Realiza una llamada telefónica usando los parámetros recibidos:

- Usa las variables ARG1=usuario, ARG2=tiempo de espera, ARG3=opciones de llamada
- Usa la aplicación `Dial()` para hacer una llamada telefónica.

2-

same => n(record),GoSub(sub_recordcallinfo,start,1(\${ARG1},\${DIALSTATUS}))

Registra el resultado de la llamada en la base de datos

- Usa las variables `${ARG1}` (Usuario (sip1 o sip2)) y `${DIALSTATUS}` (Resultado de la llamada (ANSWER, BUSY, NOANSWER, etc.))
- Usa la aplicación `GoSub` para llamar a la subrutina anterior y le pasa el usuario que llamó y el resultado de la llamada.

* record: es una etiqueta para poder saltar ahí desde otros puntos.

3-

exten => h,1,Goto(start,record)

Cuando cuelgan la llamada, salta (a la etiqueta de record) a guardar el resultado en base de datos.

- Usa la extensión h, que se ejecuta cuando se cuelga la llamada.
- Usa la aplicación Goto() para saltar a una parte del código.

```
[sub_sipdial]
; ARG1=usuario, ARG2=tiempo, ARG3=opciones
exten => start,1,Verbose(1,Dialing SIP user ${ARG1})
same => n,Dial(PJSIP/${ARG1},${ARG2},${ARG3})      ; Hacer la llamada
same => n(record),GoSub(sub_recordcallinfo,start,1(${ARG1},${DI-
ALSTATUS}))
same => n,Return()

; Si contestó y luego colgó, también guardar
exten => h,1,Goto(start,record)
```

Ejercicio 5: Implementar las extensiones 01 y 02

Implementa las extensiones 01 y 02 a partir del código 5. Para ello completa los fragmentos resaltados (a) y (b) con llamadas a la subrutina `sub_sipdial`, pasándole como parámetro el nombre del *endpoint* PJSIP correspondiente (`sip1` o `sip2`).

Una vez implementadas las extensiones y aplicados los cambios en Asterisk realiza algunas llamadas de prueba y comprueba que la información de las llamadas se registra en AstDB como se indicó en el apartado 2.3.1. Para ello ejecuta en la consola de Asterisk los comandos `database show calls_sip1` y `database show calls_sip2` y copia su salida. ¿Se registra correctamente la información de las llamadas?

IMPLEMENTAR LAS EXTENSIONES 01 Y 02. SIP1 DEBE PODER LLAMAR A 02 Y SIP2 DEBE PODER LLAMAR A 01. AMBOS USARÁN LAS SUBROUTINAS DE LOS DOS EJERCICIOS ANTERIORES PARA REGISTRAR LOS RESULTADOS DE LAS LLAMADAS.

En este ejercicio no hay que completar una subrutina, sino el contexto de [practica1] donde tenemos predefinidas las extensiones a las que tenemos que llamar.

a-

same => n,GoSub(sub_sipdial,start,1(sip1,10,))

Esta es la línea para usar 01. Llama a sip1 de manera controlada y guardar el resultado de la llamada.

- Usa la aplicación GoSub() para saltar a la subrutina del ejercicio anterior y le pasa los parámetros correspondientes.

b-

same => n,GoSub(sub_sipdial,start,1(sip2,10,))

Esta es la línea para usar 02. Llama a sip2 de manera controlada y guardar el resultado de la llamada.

- Usa la aplicación GoSub() para saltar a la subrutina del ejercicio anterior y le pasa los parámetros correspondientes.

Podemos comprobar que las extensiones 01 y 02 usando `database show calls_sip1/2`

```
freepbx*CLI> voicemail show users
Context      Mbox  User      Zone      NewMsg
sm           001   Ismael
sm           002   Ismael      0
2 voicemail users configured.
```

Parte del código correspondiente al contexto de [practica1]:

```
[practica1]

exten => 01,1,Verbose(1, Dial user sip1)
same => n,GoSub(sub_sipdial,start,1(sip1,10,))
same => n, HangUp()

exten => 02,1,Verbose(1, Dial user sip2)
same => n,GoSub(sub_sipdial,start,1(sip2,10,))
same => n, HangUp()
```

Ejercicio 6: Implementar la subrutina `sub_voicemail`

Implementa la subrutina `sub_voicemail` a partir del código 6 completando los fragmentos resaltados (a) y (b) como sigue:

1. En el fragmento resaltado (a) usa la función `IF` para establecer la variable local `message` a `b` si el parámetro `ARG2` (el valor de `DIALSTATUS`) es igual a la cadena `BUSY`, o establecerla a `u` en caso contrario.
2. En el fragmento resaltado (b) usa la aplicación `VoiceMail()` para lanzar el buzón de voz, pasándole como primer parámetro el valor de `ARG1`, y como segundo parámetro la concatenación de `message` y `ARG3`.

ESTA SUBROUTINA HACE QUE, SI NO SE CONSIGUE LLAMAR SI ACTIVE EL BUZÓN DE VOZ

ARG1=buzón, ARG2=estado, ARG3=opciones

1-

```
Set(LOCAL(message)=${IF("${ARG2}"="BUSY"?b:u)})
```

Usa la función IF() para decidir qué tipo de mensaje de buzón reproducir según el estado de la llamada. Por ejemplo si la llamada terminó con BUSY usa mensaje de ocupado y si la llamada terminó con NOANSWER, usa mensaje de no disponible

- Usa las variables `${ARG2}` (Estado de la llamada (BUSY, NOANSWER, etc.)) y `LOCAL(message)` (Nueva variable que se crea)
- Usa la función `IF()`
- Usa la expresión `"${ARG2}"="BUSY"` que Compara si ARG2 es "BUSY"

2-

```
VoiceMail(${ARG1},${message}${ARG3})
```

Permitir que los usuarios dejen mensajes de voz cuando no se puede completar la llamada. Usa VoiceMail() para activar el buzón de voz con el tipo de mensaje correcto.

```
[sub_voicemail]
; ARG1=buzón, ARG2=estado, ARG3=opciones
exten => start,1,Verbose(3,Executing sub_voicemail for ${ARG1})

; Si no hay estado o fue ANSWER, no hacer nada
same => n,GotoIf($[ ${ISNULL(${ARG2})} | ${ARG2} = ANSWER ]?exit)

; Decidir mensaje: b=ocupado, u=no disponible
same => n,Set(LOCAL(message)=${IF($[ "${ARG2}"="BUSY" ]?b:u)})

; Activar buzón de voz
same => n,VoiceMail(${ARG1},${message}${ARG3})

same => n(exit),Return()
```


Ejercicio 7: Implementar la extensión 00X

Implementa la extensión 00X. Para ello en primer lugar copia al *dialplan* las variables globales indicadas en el código 7, las cuales serán necesarias para el código de la extensión. A continuación implementa la extensión a partir del código 8 completando los fragmentos resaltados (a), (b), y (c) como sigue:

1. En el fragmento resaltado (a) usa un patrón de extensión que coincida con las extensiones 001 o 002.
2. En el fragmento resaltado (b) usa la variable de Asterisk que proporciona el número de la extensión actual para que este se reproduzca.
3. En el fragmento resaltado (c) inserta la expresión adecuada para que a la variable `user` se le asigne la cadena `sip1` si $X = 1$, o `sip2` si $X = 2$. Para esto simplemente concatena la cadena `sip` a la cadena obtenida de manipular la variable de Asterisk de la extensión actual para extraer su último dígito.

Una vez configurada la extensión prueba a dejar algún mensaje en el buzón de voz, luego ejecuta en la consola de Asterisk el comando `voicemail show users` y copia su salida. En la salida de este comando se indica para cada usuario en la columna «NewMsg» el número de mensajes nuevos en el buzón de voz. ¿Se están registrando correctamente los mensajes en el buzón de voz?

IMPLEMENTAR LAS EXTENSIONES 001 Y 002. SIP1 DEBE PODER LLAMAR A 002 Y SIP2 DEBE PODER LLAMAR A 001. AMBOS USARÁN LA SUBROUTINA DEL EJERCICIO ANTERIOR PARA ESTABLECER LOS BUZONES DE VOZ.

En este ejercicio no hay que completar una subrutina, sino el contexto de [practica1] donde tenemos predefinidas las extensiones a las que tenemos que llamar. En este caso completamos la parte para llamar a 001 y 002 que NO nos o dan hecho en [practica1].

1.-

exten => 00[1-2],1,Verbose(1,Say extension and dial user)

Define una extensión con patrón que funciona para 001 y 002 usando _00[1-2].

- Usa pattern matching en _00[1-2], patron que acepta 001 o 002.
- Usa la aplicación Verbose para mostrar un mensaje en consola cuando se teclee 001 o 002.

2.-

same => n,SayDigits(\${EXTEN})

Reproduce el número que marcó el usuario, dígito por dígito.

- Usa la aplicación SayDigits(), que dice un número por dígito.

- Usa la variable EXTEN que es una variable automática con el número marcado.

3.-

same => n, Set(user=sip\${EXTEN:-1})

Transforma el número de extensión en el nombre del usuario para poder llamarlo.

- Usa la aplicación Set().
- Usa las variables EXTEN y user.
- Usa **\${EXTEN:-1}** que toma el último dígito del número.

Convierte el número marcado en nombre de usuario:

- 001 → sip1 (último dígito: 1)
- 002 → sip2 (último dígito: 2)

```
[practica1]
...

exten => _00[1-2],1,Verbose(1,Say extension and dial user)
same => n, Playback(you-have-dialed)
same => n,SayDigits(${EXTEN})
same => n,Set(user=sip${EXTEN:-1})
same => n, GoSub(sub_sipdial,start,1(${user},10))
same => n, GoSub(sub_voicemail,start,1(${MAILBOX_${user}}},${DIALSTATUS}))
same => n, HangUp()
```

Ejercicio 8: Implementar la extensión 120

Implementa la extensión **120** a partir del código **9** completando los fragmentos resaltados (a), (b), y (c) como sigue:

1. En primer lugar, necesitamos obtener el nombre del *endpoint* que ha realizado la llamada, que en nuestro caso será **sip1** o **sip2**. Para esto usaremos la función **CHANNEL()**, con la que podemos obtener información del canal. En la sección 3.5 del documento *Notas de programación en Asterisk* tienes mas información acerca de esta función.
2. Una vez almacenado el nombre del *endpoint* en la variable **user**, en la siguiente prioridad de la extensión se guarda la dirección de buzón de voz en la variable **mailbox** a partir de la variable global correspondiente definida en el código **7**. Sin embargo, si definimos un nuevo usuario SIP y no definimos una nueva variable global con su dirección de buzón de voz, el valor de la variable **mailbox** será la cadena vacía. Completa el fragmento resaltado (b) usando la aplicación **GotoIf()** de forma que si **mailbox** es una cadena vacía se salte a la prioridad con la etiqueta **end**. Para esto revisa el apéndice **A**.
3. Finalmente, Completa el fragmento resaltado (c) para consultar los mensajes del buzón de voz indicado en en la variable **mailbox**.

Una vez configurada la extensión el usuario podrá llamar y escuchar los mensajes que se le han dejado en su buzón. Prueba a llamar a la extensión, ¿Puedes escuchar los mensajes enviados al usuario?

IMPLEMENTAR LA EXTENSIÓN 120. SIP1 U SIP2 DEBEN PODER LLAMAR A 120 AMBOS DEBEN PODER ESCUCHAR SUS BUZONES DE VOZ EN DONDE ESTÁN LOS MENSAJES VIEJOS (YA ESCUCHADOS) Y LOS NUEVOS (NO ESCUCHADOS TODAVÍA).

En este ejercicio no hay que completar una subrutina, sino el contexto de [practica1] donde tenemos predefinidas las extensiones a las que tenemos que llamar. En este caso completamos la parte para llamar a 120.

1.-

same => n,Set(user=\${CHANNEL(endpoint)})

Saber automáticamente qué usuario está usando la extensión, sin necesidad de que lo especifique.

- Usa la aplicación Set() para asignar valor a variable.
- Usa la variable user que será el endpoint que llama.
- Usa la función CHANNEL para obtener información del canal y saber que endpoint está llamando.

2.-

same => n,GotoIf("\${mailbox}" = "")?end)

Si el mailbox está vacío salta a la etiqueta end y si no continua con la siguiente línea.

- Usa la aplicación Gotoif() que es un salto condicional.
- Usa la variable \${mailbox}, que es la variable con la dirección del buzón.
- Usa la expresión del = para comparar si el mailbox está vacío.

3.- same => n,VoiceMailMain(\${mailbox})

Da acceso al usuario para gestionar sus mensajes de voz.

RESULTADO:

- Pide PIN de seguridad (si no se salta con opción s)
- Muestra menú de buzón de voz
- Permite escuchar, guardar, borrar mensajes
- Permite configurar saludos

```
exten => 120,1,Verbose(1,Check voicemail messages)
same => n,Set(user=${CHANNEL(endpoint)})
same => n,Set(mailbox=${MAILBOX_${user}})
same => n,GotoIf("${mailbox}" = "")?end)
same => n,VoiceMailMain(${mailbox})
same => n,HangUp()
same => n(end),Verbose(1,Mailbox for user ${user} not defined)
same => n,Playback(feature-not-avail-line)
same => n,HangUp()
```

Se pueden escuchar los mensajes de cada usuario, los suyos propios, cuando llama al 120 con su correspondiente contraseña 0000 (que es la que configuramos para el buzón de voz de cada usuario en el archivo voicemail.conf).

Ejercicio 9: Implementar la extensión 99X

Implementa la extensión **99X**. Para ello primero copia al dialplan la variable global **TIMEZONE** indicada en el código **10**. A continuación implementa la extensión a partir del código **11** completando los fragmentos resaltados (a), (b), y (c) como sigue:

1. El el fragmento resaltado (a) usa un patrón de extensión que coincida con las extensiones 991 o 992.
2. El el fragmento resaltado (b) usa la aplicación **GoToIfTime()** para comprobar si la llamada se produce en el horario permitido definido en la extensión **99X**. En caso de que sea así, haz que se acepte saltando a la prioridad con la etiqueta **accept**.
3. Finalmente, en el fragmento resaltado (c) llama a la subrutina **sub_sipdial** pasandole como usuario **sip1** si $X = 1$, o **sip2** si $X = 2$.

IMPLEMENTAR LAS EXTENSIONES 991 Y 992. SIP1 DEBE PODER LLAMAR A 992 Y SIP2 DEBE PODER LLAMAR A 991, PERO SOLO EN UNA FRANJA HORARIA DETERMINADA. SOLO DEBE ACEPTAR LLAMADAS ENTRE LAS 15:30 Y LAS 20:00 DURANTE TODOS LOS MESES MENOS AGOSTO. SI SE LLAMA FUERA DEL HORARIO SE REPRODUCE UN MENSAJE DE ERROR. SI SE LLAMA EN EL HORARIO PERMITIDO SE REALIZA UNA LLAMADA NORMAL.

En este ejercicio no hay que completar una subrutina, sino el contexto de [practica1] donde tenemos predefinidas las extensiones a las que tenemos que llamar. En este caso completamos la parte para llamar a 991 y 992.

1.-

exten => _99[1-2],1,Verbose(1,Accept calls only in the given times)

Define una extensión con patrón que se activa cuando se marcan:

- **991** → Para llamar a sip1 (solo en horario permitido)
- **992** → Para llamar a sip2 (solo en horario permitido)

- Usa pattern matching para poder especificar si llamar a 991 o 992.
- Usa la aplicación Verbose para mostrar un texto en consola.

2.-

```
same => n,GoToIfTime(15:30-20:00,mon-thu,*,jan-jul&sep-dec,{TIMEZONE}?accept)
```

Verifica si la llamada está dentro del horario permitido. El horario permitido es de lunes a jueves de 15:30 a 20:00 todos los meses salvo agosto y en la zona de TIMEZONE.

3.-

```
same => n(accept),GoSub(sub_sipdial,start,1(sip${EXTEN:2},10))
```

Si la anterior condición se cumplió salta a esta línea ya que contiene la etiqueta de accept. Aquí se salta a la subrutina de subsipdial que es aquella que realiza una llamada de forma normal

```
exten => _99[1-2],1,Verbose(1,Accept calls only in the given times)
same => n,Verbose(1,System time is: ${STRFTIME()})
same => n,Verbose(1,Local time is: ${STRFTIME(,{TIMEZONE})})
same => n,GoToIfTime(15:30-20:00,mon-thu,*,jan-jul&sep-dec,{ TIME-
ZONE}?accept)
same => n,Playback(unavailable)
same => n,Playback(please-try-again-later)
same => n,Goto(exit)
same => n(accept),GoSub(sub_sipdial,start,1(sip${EXTEN:2},10))
same => n(exit), HangUp()
```

Ejercicio 10: Implementar la subrutina **sub_conference**

Implementa la subrutina **sub_conference** a partir del código 12 como sigue:

1. En el fragmento resaltado (a) usa la aplicación **Read()** para solicitar al usuario que introduzca un PIN de 4 dígitos y que suene el sonido **beep** antes de hacerlo. En la sección 3.4.3 del documento *Notas de programación en Asterisk* se explica en detalle el funcionamiento de **Read()**. Asegúrate de entender bien su funcionamiento antes de continuar.
2. En el fragmento resaltado (b) añade la lógica para verificar si el PIN introducido es correcto. Si es correcto, salta a la etiqueta **auth_success**, si no, reproduce un mensaje de error (**conf-invalidpin**) y vuelve al bucle de autenticación.
3. En el fragmento resaltado (c) haz que el usuario se conecte a la sala de conferencia número 7007 usando **ConfBridge()**.

ESTA SUBROUTINA HACE QUE LOS USUARIOS SE PUEDAN CONECTAR A UNA CONFERENCIA MEDIANTE UN PIN.

1.-

same => n,Read(LOCAL(input_pin),beep,4)

Lee hasta 4 dígitos que marque el usuario y los guarda en LOCAL(input_pin)

Read(variable, audio, max_dígitos, opciones, intentos, timeout)

- Usa la aplicación Read para leer una variable LOCAL(input_pin), que es una variable temporal donde se guardan los dígitos. Tiene también de parámetros beep, que es un audio que se reproduce ante de leer y 4 que indica el máximo de dígitos a leer.

2.-

same => n,GotoIf("\${LOCAL(input_pin)}" = "\${LOCAL(correct_pin)}"?auth_success)

same => n,Playback(conf-invalidpin)

same => n,Goto(auth_loop)

Compara el PIN ingresado con el PIN correcto. Si son iguales salta a la etiqueta auth_sucess. Si son diferentes continúa en la siguiente línea. Si es incorrecto reproduce un audio de PIN inválido y en la siguiente línea vuelve al inicio de la autenticación. Permite intentarlo nuevamente.

3.-

same => n,ConfBridge(7007)

Conecta al usuario a la sala de conferencia número 7007

- Usa la aplicación ConfBridge que conecta una sala de conferencia. Usa el parámetro 7007 que es el número de la sala de conferencia.

```
[sub_conference]
; Subroutine to enter a conference room
; ARG1: Required PIN
; ARG2: Maximum attempts

exten => start,1,Verbose(1,Starting conference join process)
same => n,Set(LOCAL(correct_pin)=${ARG1})
same => n,Set(LOCAL(max_attempts)=${ARG2})
same => n,Set(LOCAL(attempts)=0)

same => n(auth_loop),Set(LOCAL(attempts)=[${LOCAL(attempts)} + 1])
same => n,GotoIf([${LOCAL(attempts)} > ${LOCAL(max_at-
tempts)}])?auth_failed)
same => n,Playback(conf-getpin)
same => n,Read(LOCAL(input_pin),beep,4)
same => n,GotoIf(["${LOCAL(input_pin)}" = "${LOCAL(cor-
rect_pin)}"]?auth_success)
same => n,Playback(conf-invalidpin)
same => n,Goto(auth_loop)

same => n(auth_success),Verbose(1,Auth success)
same => n,Playback(pin-number-accepted)
same => n,ConfBridge(7007)
same => n,Hangup()

same => n(auth_failed),Verbose(1,Auth failed)
same => n,Playback(conf-kicked)
same => n,Hangup()
```


Ejercicio 11: Implementar la extensión 300

Implementa la extensión 300 a partir del código 13 completando el fragmento resaltado (a) como sigue:

1. En el fragmento resaltado (a) llama a la subrutina `sub_conference` pasándole como parámetros el PIN requerido (en este caso, 4323) y el número máximo de intentos (en este caso, 3).

COMPLETAR LA EXTENSIÓN 300 QUE PERMITE UNIRSE A UNA CONFERENCIA.
UTILIZA LA SUBROUTINA ANTERIOR `sub_conference`.

1.-

same => n,GoSub(sub_conference,start,1(4323,3))

Salta a la subrutina anterior con la prioridad 1 de la etiqueta start y con los parámetros PIN y max_attempts que es el número máximo de intentos para poner bien el PIN antes de que se cuelgue la llamada.

Ejercicio 12: Implementar la subrutina `sub_getcallscount`

Implementa la subrutina `sub_getcallscount` a partir del código 14 como sigue:

1. En el fragmento resaltado (a) guarda en `LOCAL(familykey)` la ruta de la base de datos que almacena el valor pedido.
2. En el fragmento resaltado (b), usa la función `DB` para extraer el dato almacenado en la ruta `LOCAL(familykey)`.
3. Por último, en el fragmento resaltado (c) haz que la subrutina devuelva el dato. En la sección 3.4.5 del documento *Notas de programación en Asterisk* se explica el mecanismo por el que una subrutina puede devolver un resultado y como se obtiene este desde la extensión que hizo la llamada.

EXÁMENES:

1. Dada la extensión:

exten => _6[1-468],1,verbose(1,Extension con patrón)

Con que números podremos llamar a esta extensión?

- a) todos los números entre 61 y 6468.
- b) 61 o 6468.
- c) todos los números entre 61 y 468
- d) 61, 62, 63, 64, 66, o 68.

d)

El patrón _6[1-468] se divide así:

- _6: El número debe empezar con "6".
- [1-468]: El siguiente dígito debe ser **uno** de los números dentro del corchete. La definición [1-468] significa un rango de 1 a 4 y los números 6 y 8 sueltos. Es decir, los dígitos válidos son: 1, 2, 3, 4, 6, 8.

Por lo tanto, los números con los que se puede llamar son: 61, 62, 63, 64, 66, 68

2. Dado el siguiente código:

exten => 100,1,Set(Count=10)

same => n,Set(Count=\${Count}+1)

same => n,Verbose(\${Count})

Que imprime la aplicación Verbose() por pantalla?

- a) 10+1
- b) res
- c) Count + 1
- d) 11

a)

En la sintaxis tradicional del dialplan de Asterisk, la línea

Set(Count=\${Count}+1) no realiza una suma matemática. En su lugar, concatena el valor de \${Count} (que es "10") con la cadena de texto "+1". Así, la variable Count queda con el valor **"10+1"**, y eso es lo que imprime Verbose().

Para que de 11 tiene que ser Count=\${\${Count}+1}). Si no hay \${...}, no se evalúa, solo concatena

3. Si tenemos la siguiente subrutina:

[subVoiceMail]

exten => start,1,VoiceMail(\${ARG1}\$default,\${ARG2})

¿Cómo podemos invocarla desde otro punto del dialplan para que salte el buzón de voz con mensaje de usuario ocupado?

Se asume que el identificador del buzón de voz coincide con la extensión asignada para llamar al usuario (p.ej: sip1 → 1001).

a) GoSub(subVoiceMail,1(\${EXTEN},\${CONTEXT}))

b) Goto(subVoiceMail,start,1)

c) GoSub(subVoiceMail,start,1(\${EXTEN},b))

d) No se puede llamar a la subrutina, ya que la extensión start es una extensión especial de Asterisk y no se puede utilizar en subrutinas.

c)

Para saltar el buzón de voz por "usuario ocupado", el segundo argumento (\${ARG2}) para la aplicación VoiceMail debe ser b. La sintaxis correcta para llamar a una subrutina es GoSub(contexto, extensión, prioridad(argumento1, argumento2)). En este caso, se le pasa la extensión actual \${EXTEN} como el buzón y b como la razón.

4. ¿Cuál de las siguientes no es una invocación de la aplicación Dial() válida en Asterisk?

a) Dial(PISIP/1001,30)

b) Dial(PISIP/1001)

c) Dial(PISIP/1001&PISIP/1002)

d) Dial(30,PISIP/1001)

d)

El orden de los parámetros en Dial() es **Dial(tecnología/destino, tiempo_timeout, opciones, URL)**. En la opción d) se pone primero el tiempo de timeout (30) y luego el destino, lo cual es inválido.

5. ¿Cómo podemos hacer para que se redirijan todas las llamadas que empiezan por 0 a la extensión que indican los tres siguientes dígitos que vienen a continuación del 0?

- a) exten => _[0]XXX, 1, Goto(\${EXTEN:0:3},1)
- b) exten => _0XXX, 1, Goto(\${EXTEN:2:4},1)
- c) exten => _0XXX, 1, Goto(\${EXTEN:2:3},1)
- d) exten => _[0]XXX, 1, Goto(\${EXTEN:1},1)

d)

El enunciado pide redirigir las llamadas que comienzan con el dígito 0 hacia la extensión formada por los tres dígitos siguientes. Por ejemplo, si se marca "0123", se debe redirigir a la extensión "123". La opción correcta es la **d)** porque utiliza el patrón `_[0]XXX` para capturar cualquier número de cuatro dígitos que empiece por 0, y luego aplica `${EXTEN:1}` que elimina precisamente ese primer dígito (el 0) y toma el resto de la cadena (los tres dígitos siguientes), redirigiendo finalmente la llamada a esa nueva extensión mediante la aplicación `Goto`.

6. ¿Qué necesitamos añadir en el recuadro indicado en la subrutina del siguiente código para que cuando se llame a la extensión 800 se reproduzca el saldo de la cuenta?

```
exten => 800,1,Answer()  
same => n,GoSub(sub_setbalance,start,1())  
same => n,Playback(account-balance-is)  
same => n,SayNumber(${balance})  
...  
[sub_setbalance]  
exten => start, 1, Playback(hello-world)  
same => n,Set(balance=100)  
same => n, _____
```

- a) Return()
- b) Hangup()
- c) No sería necesario añadir nada, sobraría la segunda instrucción en la subrutina.
- d) Goto(800,1)

a)

La aplicación GoSub llama a una subrutina. Para que el control vuelva a la línea siguiente de donde se llamó (es decir, a Playback(account-balance-is)), la subrutina debe terminar con Return(). Si se usara Hangup(), la llamada colgaría en la subrutina y no se reproduciría el saldo.

7. ¿Qué hace la siguiente instrucción?

same => n, Goto(start,1)

- a) Da un error, ya que en Asterisk es necesario establecer una condición para realizar un salto.
- b) Salta a la prioridad 1 de la extensión start
- c) Salta a la prioridad start de la extensión 1
- d) Salta a la extensión 1 del contexto start

b)

La sintaxis Goto(start,1) significa: ve a la extensión llamada start, a su prioridad número 1, dentro del **mismo contexto** actual.

8. ¿Para qué se usa el fichero extensions_custom.conf?

- a) Para configurar los módulos que se cargan al arrancar el servidor de Asterisk.
- b) Para definir los parámetros de configuración del servidor de Asterisk
- c) Para configurar los clientes SIP en Asterisk.
- d) Para programar el dialplan.

d)

9. ¿Cual es el código correcto para permitir llamadas solo de lunes a viernes de 9:00 a 14:00 y de 16:00 a 18:00?

a. exten => 600, 1, Answer()

same => n, GotoIfTime(9:00-14:00 & 16:00-18:00, mon-fri, *, *?:allow)

same => n, Hangup()

same => n(allow),...

b. exten => 600, 1, Answer()

same => n, GotoIfTime(9:00-14:00 & 16:00-18:00, mon-fri, *, *?:allow:)

same => n, Hangup()

same => n(allow),...

c. exten => 600, 1, Answer()

same => n, GotoIfTime(9:00&14:00 - 16:00&18:00, mon&fri, *, *?:allow:)

same => n, Hangup()

same => n(allow),...

d. exten => 600, 1, Answer()

same => n, GotoIfTime(9:00&14:00 - 16:00&18:00, mon&fri, *, *?:allow)

same => n, Hangup()

same => n(allow),...

a)

Como tal es la menos incorrecta pero tampoco es correcta del todo.

```
exten => 600,1,Answer()  
same => n,GotoIfTime(9:00-14:00|16:00-18:00,mon-fri,*,*?:allow:)  
same => n,Hangup()  
same => n(allow),...
```

10. Si tenemos una variable numérica COUNT, ¿Cuál sería la forma correcta de sumarle 1?

a) same => n,Set(COUNT=\${COUNT+1})

b) same => n,Set(COUNT=\${\${COUNT}+1})

- c) same => n,Set(COUNT=\${COUNT}+1)
- d) same => n,Set(\${COUNT}=\${COUNT}+1)

b)

Para sumar 1 a una variable numérica en Asterisk, hay que usar la sintaxis de operaciones `$[...]` y poner la variable entre `${}` para que se interprete su valor real. Por eso la forma correcta es `Set(COUNT=${COUNT}+1)`, porque así Asterisk entiende que debe leer el valor actual de COUNT, sumarle 1 y guardar el resultado. Las otras opciones o bien tratan la variable como texto o usan una sintaxis inválida.

11. ¿Cuál sería la forma correcta de asignarle a la variable de canal NewCount el valor de la variable Count?

- a) Set(NewCount=\${Count})
- b) SetVar(\${NewCount}=\${Count})
- c) Set(NewCount=Count)
- d) Set(\${NewCount}=\${Count})

a)

La aplicación Set se usa para asignar valores. A la izquierda del igual va el **nombre** de la nueva variable, y a la derecha el valor, que si es de otra variable, se referencia con `${}`.

12. Dado el siguiente código del dialplan:

same => n, Set(res=\${IF(\${X} < 5)?1:2})

same => n, Verbose(\${res})

Si el contenido de X es 10, ¿qué imprimirá la aplicación Verbose?

- a) No imprimirá nada
- b) 2
- c) res
- d) 1

b)

$\$X = 10$

Condición: $[\$X < 5] \rightarrow 10 < 5 \rightarrow \text{falso}$

IF(condición?valor_si_verdadero:valor_si_falso) $\rightarrow$ devuelve **2**

Verbose($\{res\}$) imprime el valor de res, que ahora es **2**

13. ¿Qué hace la siguiente instrucción?

Same => n,Goto(start)

- a) Da un error, ya que en Asterisk es necesario establecer una condición para realizar un salto.
- b) Salta a la extensión start del contexto actual.
- c) Salta a la subrutina start
- d) Salta a la prioridad con la etiqueta start en la extensión actual

d)

14. La aplicación SayNumber nos permite...

- a) Reproducir un número dígito a dígito en el canal.
- b) Enviar un número a un destino.
- c) Reproducir un número en el canal.
- d) Guardar un número en la base de datos.

c)

15. La aplicación SayDigits nos permite...

- a) Enviar un número a un destino.
- b) Guardar un número en la base de datos.
- c) Reproducir un número en el canal.
- d) Reproducir un número dígito a dígito en el canal.

d)

16. ¿Para qué se usa el fichero pjsip_custom.conf?

- a) Para programar el dialplan.
- b) Para configurar los módulos que se cargan al arrancar el servidor de Asterisk.
- c) Para configurar los clientes SIP en Asterisk.
- d) Para definir los parámetros de configuración del servidor de Asterisk.

c)

17. ¿Qué representa el elemento test en este código?

Same => n(test), Verbose(Prueba)

- a) Una extensión.
- b) Una etiqueta para un salto. // Una referencia para un salto.
- c) Un contexto
- d) Un canal.

b)

La sintaxis n(test) define una **etiqueta** llamada "test" en esa prioridad. Esta etiqueta puede ser usada como destino en un Goto(test) para saltar a esa parte del dialplan.

18. Dado el siguiente código

```
exten => 289,1,Answer()  
same => n, Gotolf($[ ${X} = 1 ]?other)  
same => n, Playback(digits/1)  
same => n, Hangup()  
same => n(other), Playback(digits/2)  
same => n, Hangup()
```

Si el contenido de la variable X es 1, ¿qué hace el código?

- a) Termina la llamada sin reproducir nada.
- b) Reproduce el dígito 2 y termina la llamada
- c) Reproduce los dígitos 1 y 2 y termina la llamada.
- d) Reproduce el dígito 1 y termina la llamada.

b)

$\${X} = 1$

Gotolf($\${X} = 1$]?other) → verdadero, salta a la etiqueta other

En other: Playback(digits/2) → reproduce el dígito 2

Luego: Hangup() → termina la llamada

19. Si tenemos una variable numérica COUNT, ¿cómo podemos comprobar si el valor de COUNT es mayor que el valor de otra variable N, guardando el valor de la comparación en la variable res?

- a) same => n, Set(res= $\${COUNT > N}$)
- b) same => n, Set(res= $\${COUNT} > \${N}$)

c) same => n, Set(\$[res]=\$[COUNT]>\$[N])

d) same => n, Set(res=\$[COUNT]>\$[N])

b)

La opción correcta es Set(res=\${COUNT}>\${N}) (lo que en el examen aparece como b) porque en Asterisk, para hacer comparaciones numéricas, las variables deben expandirse con \${} y la expresión completa se evalúa dentro de [...]. Esto permite que el valor de COUNT se compare con el de N, y el resultado de la comparación (1 si es verdadero, 0 si es falso) se asigne a la variable res. Sin \${}, Asterisk no interpretaría correctamente las variables y la operación no funcionar

20. Dado el siguiente código del dialplan ¿Qué ocurre en este caso si un usuario definido en el contexto [menu] marca 200 y luego introduce la extensión 3?

[menu]

exten => 200, 1, Playback(welcome)

same => n, WaitExten(10)

exten => _N, 1, Playback(digits/\${EXTEN})

same => n, Hangup()

exten => t, 1, Playback(an-error-has-occurred)

same => n, Hangup()

exten => i, 1, Playback(pbx-invalid)

same => n, Goto(200,1)

Seleccione unha:

a. Se reproduce el audio del dígito 3 y se termina la llamada.

b. Se reproduce un mensaje de error y se termina la llamada.

c. Se reproduce el audio del dígito 3 y se queda esperando por una nueva extensión.

d. Se reproduce un mensaje de error y se vuelve a esperar por una nueva extensión.

a)

Cuando un usuario marca **200**, la llamada entra en la extensión 200 y se reproduce el mensaje welcome. Después, la aplicación WaitExten(10) espera hasta 10 segundos a que el usuario introduzca otra extensión. Si el usuario marca **3**, esta entrada coincide con la extensión _N, que captura cualquier dígito único. Entonces se ejecuta Playback(digits/3), reproduciendo el audio correspondiente al dígito 3, y a continuación se ejecuta Hangup(), finalizando la llamada. No se produce ningún mensaje de error ni se vuelve a esperar por otra extensión, porque la ejecución sigue la secuencia definida en el dialplan y la llamada termina tras el Hangup()

21. La extensión especial "i" se utiliza para...

- a) Manejar el caso de que se reciba una llamada a una extensión desconocida.
- b) Manejar el caso de que el usuario introduzca una extensión inválida en un menú interactivo
- c) Manejar una llamada establecida sin respuesta del destinatario.
- d) Manejar el caso de que el usuario no introduzca ninguna extensión en un menú interactivo

b)

En Asterisk, la extensión especial **i** se activa cuando un usuario marca un número que **no coincide con ninguna extensión definida** en el contexto actual, generalmente dentro de un menú interactivo. Por ejemplo, si el menú espera los dígitos 1 o 2 y el usuario marca 5, la ejecución se desviará a la extensión i

EXTENSIONES:

Letra/Patrón	Nombre
g	go on (continua la llamada)
i	invalid (inválida)
t	timeout (sin dígito)
h	hangup (colgar)
s	start (por defecto)
_	patrón (wildcard)

22. Dado el siguiente código:

```
exten => 100, 1, Set(word1=hello)
same => n, Set(word2=world)
same => n, Set(res=${word1}+${word2})
same => n, Verbose(${res})
```

- a) hello+world
- b) hello world
- c) res
- d) word1word2

a)

Al igual que en la pregunta 2, la línea `Set(res=${word1}+${word2})` concatena los valores de las variables con el símbolo `+` en medio de forma literal. El resultado es "hello+world".

Para concatenar texto hay que usar `$`

23. ¿Para qué se usa el fichero `extensions.conf`?

- a) Para programar el dialplan.
- b) Para definir los parámetros de configuración del servidor de Asterisk
- c) Para configurar los módulos que se cargan al arrancar el servidor de Asterisk.
- d) Para configurar las opciones de los usuarios en Asterisk.

24. ¿Qué hace la siguiente ejecución de la aplicación VoiceMailMain()?

exten => 100,1,VoiceMailMain()

- a) Pregunta por un número de buzón y permite dejar un mensaje en él.
- b) Pregunta por un número de buzón de voz y permite acceder para escuchar los mensajes recibidos.
- c) Permite dejar un mensaje en el buzón de voz de la extensión de destino de la llamada.
- d) Permite acceder al buzón de voz de la extensión de origen de la llamada para escuchar los mensajes recibidos.

b)

VoiceMailMain() es la aplicación que se usa para que un usuario **acceda** a su buzón de voz, introduciendo su número y su PIN, para escuchar, gestionar y grabar su saludo. No es para dejar un mensaje (eso se hace con VoiceMail).

25. Dado el siguiente código:

same => n, GotolTime(08:00-15:00, tue-sat, *, jan-jul@sep-dic?a,b,1)

Si la fecha es el martes 15 de julio a las 13:00, ¿Cuál de las siguientes es correcta?

- a) Salta al contexto a
- b) Salta a la extensión a
- c) Salta a la extensión 1
- d) Salta al contexto b

a)

26. Tenemos una variable numérica TEST y queremos programar en el dialplan una línea que salte a la extensión 10 si TEST contiene el valor 123, o de lo contrario que salte a la extensión 20, ¿Cuál de las siguientes opciones es correcta?

- a. same => n, GotoIf(\$[TEST=123]?10:20)
- b. same => n, GotoIf(\$[\$[TEST]=123]?10,1:20,1)
- c. same => n, GotoIf(TEST=123?10,1:20,1)
- d. same => n, GotoIf(\$[\$[TEST]=123]?1,10:1,20)

a)

EXTRA

1. Dada la siguiente extensión con patrón:

exten => _5[2-5]X,1,Verbose(1,Extensión con patrón)

¿Con qué números podremos llamar a esta extensión?

- a) 52, 53, 54, 55
- b) 520, 521, 522, ..., 559
- c) 520-529, 530-539, 540-549, 550-559
- d) 52X, 53X, 54X, 55X donde X es cualquier dígito

c)

2. Dado el siguiente código:

exten => 200,1,Set(Counter=5)

same => n,Set(Counter=\${Counter}+2)

same => n,Verbose(\${Counter})

¿Qué imprime la aplicación Verbose()?

- a) 5+2
- b) 7
- c) Counter+2
- d) 52

a)

Para que fuse 7 tendría que ser:

same => n,Set(Counter=\${\${Counter} + 2])

Para concatenar se usa \$

3. Si tenemos la siguiente subrutina:

[subDialUser]

exten => start,1,Dial(PJSIP/\${ARG1},\${ARG2})

same => n,Return(\${DIALSTATUS})

¿Cómo la invocamos correctamente para llamar al usuario "sip3" con timeout de 15 segundos?

- a) GoSub(subDialUser,sip3,15)
- b) GoSub(subDialUser,start,1(sip3,15))
- c) Goto(subDialUser,start,1)
- d) Dial(PJSIP/sip3,15)

b)

4. ¿Cuál de las siguientes NO es una invocación válida de la aplicación Dial()?

- a) Dial(PJSIP/1001,20)
- b) Dial(PJSIP/1001&PJSIP/1002,30)
- c) Dial(20,PJSIP/1001)
- d) Dial(PJSIP/1001,,m)

c)

5. Dado el siguiente código:

text

exten => 500,1,Set(VAR1=hello)

same => n,Set(VAR2=world)

same => n,Set(RESULT=\${VAR1} \${VAR2})

same => n,Verbose(\${RESULT})

¿Qué se imprime?

- a) hello+world
- b) hello world
- c) RESULT
- d) VAR1VAR2

b)

6. La aplicación VoiceMailMain() se utiliza para:

- a) Dejar mensajes en buzones de voz
- b) Consultar mensajes recibidos
- c) Configurar buzones nuevos
- d) Eliminar mensajes antiguos

b)

7. ¿Qué hace esta instrucción?

same => n,Gotof(\$[COUNT] > 10]?high:low)

- a) Error de sintaxis
- b) Salta a 'high' si COUNT > 10, a 'low' si no
- c) Compara COUNT con las cadenas 'high' y 'low'
- d) Asigna el valor 10 a COUNT

b)

8. ¿Qué aplicación se usa para reproducir audio que puede ser interrumpido por dígitos DTMF?

- a) Playback()
- b) SayDigits()
- c) Background()
- d) Read()

c)

9. ¿Cómo se define correctamente una variable local en una subrutina?

- a) Set(VAR=valor)
- b) Set(GLOBAL(VAR)=valor)
- c) Set(LOCAL(VAR)=valor)
- d) SetVar(VAR=valor)

c)

10. La variable DIALSTATUS puede contener:

- a) ANSWER, BUSY, NOANSWER, CHANUNAVAIL
- b) SUCCESS, FAILED, TIMEOUT
- c) CONNECTED, DISCONNECTED, RINGING
- d) TRUE, FALSE, UNKNOWN

a)

11. ¿Cómo se accede al valor devuelto por una subrutina?

- a) \${RETURN_VALUE}
- b) \${SUB_RETVAL}
- c) \${GOSUB_RETVAL}
- d) \${ARG1}

c)

12. ¿Qué hace esta instrucción?

same =>

```
n,Set(DB(calls/${EXTEN}/answered)=$[${DB(calls/${EXTEN}/answered)} + 1])
```

- a) Lee un valor de la base de datos
- b) Incrementa un contador en AstDB
- c) Crea una nueva tabla en la base de datos
- d) Borra un registro de la base de datos

b)

13. Sobre la configuración SIP en pjsip_custom.conf:

¿Qué parámetro en una sección de tipo endpoint define el contexto del dialplan que se ejecutará para ese cliente?

- a) context
- b) dialplan
- c) extension
- d) route

a)

14. Dada la siguiente variable:

```
${numero} = "123456789"
```

¿Qué valor devuelve \${numero:2:4}?

- a) "3456"
- b) "2345"
- c) "345"
- d) "234"

a)

Esto indica empezar en el índice 2 y tomar 4 caracteres a partir de ahí. No quiere decir tomar desde las posiciones 2 a 4.

15. En expresiones de Asterisk, ¿qué significa el operador :?

- a) División
- b) Concatenación
- c) Comparación de expresiones regulares
- d) Asignación

c) Comparación de expresiones regulares

El operador : compara dos expresiones regulares, asumiendo que la expresión regular ya lleva un ^ implícito (debe coincidir desde el principio del string).

16. ¿Cuál es la diferencia principal entre Playback() y Background()?

- a) Playback() permite interrupciones por DTMF, Background() no
- b) Background() permite interrupciones por DTMF, Playback() no
- c) Ambas son iguales
- d) Playback() es para música, Background() para voz

b)

17. Sobre la aplicación Read():

¿Qué variable se establece automáticamente con el resultado de la lectura?

- a) READSTATUS
- b) INPUT_STATUS
- c) READ_RESULT
- d) DTMF_STATUS

a)

18. Dada esta instrucción:

same => n,GotoIf(\$[\${EXTEN} = 100 | \${EXTEN} = 200]?option1:option2)

Si EXTEN = 150, ¿a dónde saltará?

- a) option1
- b) option2
- c) Dará error
- d) Continuará en la siguiente prioridad

b)

19. ¿Qué hace la siguiente instrucción?

same => n,Set(__PERSISTENT_VAR=valor)

- a) Crea una variable local
- b) Crea una variable que persiste solo en el siguiente salto
- c) Crea una variable que persiste en todos los saltos desde su definición
- d) Crea una variable global

c)

20. ¿Cuál es la sintaxis correcta para usar una función que devuelve un valor?

- a) FUNCTION(arg1,arg2)
- b) FUNCTION(arg1,arg2)
- c) \${FUNCTION(arg1,arg2)}
- d) {FUNCTION(arg1,arg2)}

c)

11. Dado este código:

```
exten => 300,1,Set(TIME=${STRFTIME(,,%H:%M)})
```

```
same => n,Verbose(Hora actual: ${TIME})
```

¿Qué se imprimirá si son las 14:30?

- a) "Hora actual: 14:30"
- b) "Hora actual: \${TIME}"
- c) "Hora actual: STRFTIME"
- d) Error de sintaxis

a)

12. ¿Qué aplicación se usa para ejecutar una parte del dialplan como subrutina y poder volver?

- a) Goto()
- b) GoSub()
- c) Jump()
- d) Execute()

b)